



TOBB ETÜ
Ekonomi ve Teknoloji Üniversitesi

TOBB EKONOMİ VE TEKNOLOJİ ÜNİVERSİTESİ
Bilgisayar / Yapay Zekâ Mühendisliği
BİL 401

Final Project Report

A Simulated Flash-Sale Distributed Data Pipeline: Kafka, Spark, and Citus on Docker Compose

Big Data and Distributed Data Processing

Source code and full reproduction instructions:

<https://github.com/Metekilic0/Flash-sale-system-project>

Takım Üyeleri	Mete KILIÇ- 231401028 İsmail Emre YILDIZ- 221401006
Akademik Dönem	2025–2026 Yaz Dönemi

Table of Contents

1. Abstract.....	3
2. Introduction	3
3. Related Work	4
4. System Architecture	4
4.1 Synthetic Data Generator	5
4.2 Kafka Layer	5
4.3 Stream Processing (Spark Structured Streaming)	5
4.4 Storage Layer (Citus).....	6
4.5 Monitoring.....	6
4.6 Why Docker Compose and not Kubernetes	6
5. Implementation Details	6
5.1 Anomaly-Detection Rule	6
5.2 Sharding Strategy.....	6
5.3 Synthetic Data Design	6
6. Experimental Setup.....	7
7. Results	7
7.1 End-to-End Correctness	7
7.2 Fault Tolerance – Kafka.....	8
7.3 Fault Tolerance – Citus.....	8
7.4 Scalability – Spark	9
7.5 Scalability – Citus	10
7.6 Stress Test – 5× Overload	10
7.7 Connectivity Testing	11
8. Discussion and Limitations.....	11
8.1 The anomaly threshold was not rate-aware – and we went back and fixed it.....	11
8.2 Single points of failure we accepted on purpose	13
8.3 What we would do with more time	13
9. Practical Lessons Learned	13
10. Conclusion.....	14
11. References.....	14
12. Appendix.....	15
A.1 Repository / Submission Structure	15
A.2 Spark Worker Cluster at 3 Workers (Evidence)	15
A.3 Sample Analytics on the Accumulated Dataset	15
A.4 Reproducing These Results	17
A.5 Verification Queries for the Anomaly-Detection Fix (Section 8.1).....	18

1. Abstract

Flash sales – short, high-intensity sales windows where traffic spikes by one or two orders of magnitude in seconds – are a genuinely hard problem for backend systems, partly because of the raw volume and partly because they attract scalper bots trying to buy everything before real customers can. This project builds a small, self-contained simulation of that scenario: a synthetic clickstream generator feeds an Apache Kafka cluster, Apache Spark Structured Streaming consumes the stream and separates apparent bot traffic from legitimate traffic in near real time, and the results land in Citus, a horizontally-sharded PostgreSQL cluster, where they are visualized through Prometheus and Grafana.

The whole stack – 17 containers in total – runs on a single development laptop via Docker Compose. Over the course of the project we deliberately broke parts of it on purpose: we killed Kafka brokers (one at a time and two at a time), killed a Citus worker mid-write, scaled Spark from one worker to three, added and rebalanced a third Citus worker, and finally hit the whole pipeline with five times its nominal target load just to see what would give first. None of these tests were purely theoretical – every number quoted in Section 7 comes from an actual run on our machine, logged and queried back out of the system, not estimated.

The headline results are that the system tolerates the failures it was configured to tolerate (and fails loudly, not silently, once that configuration's limits are exceeded), that scaling Spark and Citus each produced a measurable, repeatable improvement, and – somewhat to our own surprise – that the actual bottleneck under extreme load turned out to be our own single-process Python data generator rather than any of the three distributed systems we set out to test. We also found a genuine weakness in our anomaly-detection logic under backlog conditions; when we went back and queried the roughly 93 million events the system had accumulated over its multi-day lifetime (Appendix A.3), it turned out this was not a rare edge case at all – 84% of every event we ever processed had been flagged as “bot-like.” Rather than just reporting that number, we traced it to two separate causes, fixed both, and re-verified against the generator's own known ground truth: the corrected pipeline now flags almost exactly the 15% bot ratio it was actually seeded with (Section 8.1, Figure 6).

2. Introduction

E-commerce flash sales create two related but distinct engineering problems. The first is purely about scale: a system that comfortably handles a few hundred requests per second needs to survive a burst that is 50–100 times larger, for a short but unpredictable window. The second is about abuse: the same burst conditions are exactly when scalper bots are most active, because the financial incentive to buy limited-stock items before anyone else is highest. A system that scales but cannot tell bots from customers is not actually solving the business problem, even if it stays up.

For the Systems Track, our brief was to integrate at least three or four distinct open-source distributed technologies, deploy them with Docker/Docker Compose so the whole thing simulates a distributed environment on a single laptop, and then actually test – not just claim – scalability, connectivity, and fault tolerance. We chose to build a (greatly simplified) flash-sale pipeline as the vehicle for that, because it naturally needs an ingestion buffer (Kafka), a stream processor that can apply business logic in near real time (Spark), and a place to land high-volume writes that can be sharded across nodes (Citus) – three technologies with genuinely different jobs, rather than three interchangeable databases.

If we are being honest about how we landed on this exact combination: the project brief's own list of inspirations mentions “Distributed Postgresql scalability tests – Citus” and “Spark – Kafka Connection, scaling” as two separate bullet points, and we mostly picked this project because combining both into a single pipeline sounded more interesting than doing either one in isolation. In hindsight that turned out to be the harder path – three technologies that

all have to agree on hostnames, ports, and restart timing give you genuine integration bugs that a single-technology demo simply does not, and Section 9 is largely a record of the ones we hit.

This report covers, in order: the related work that shaped a few of our design choices, the system architecture as actually deployed, implementation details worth calling out specifically (the anomaly-detection rule and the sharding scheme), the experimental setup, the full set of fault-tolerance, scalability, and stress-test results, a discussion of the limitations we found (including ones we did not expect to find), the practical lessons learned getting all of this to run together, and a conclusion.

3. Related Work

The overall pattern – a log-structured broker buffering writes for a stream processor, which in turn writes into a horizontally-scaled store – is a long-standing one in industry, going back to Kafka's original purpose as LinkedIn's internal commit log for decoupling producers and consumers. Spark Structured Streaming's micro-batch execution model is one of the more common ways to bridge declarative streaming with batch-style custom logic; we specifically needed its `foreachBatch` sink rather than the purely declarative streaming API, because per-batch IP-counting and upsert-style aggregate writes are not naturally expressible as a single streaming query.

Citus turns a regular PostgreSQL cluster into a sharded, horizontally-scalable database while keeping standard SQL, and is used in production by a number of SaaS companies for high-write, time-series-like workloads – a coordinator node routes and plans queries, while a configurable number of worker nodes each hold a slice (shard) of every distributed table. We picked Citus specifically because (unlike, say, adding read replicas) it lets us demonstrate genuine horizontal write scaling and shard rebalancing, which felt closer to the spirit of the “distributed Postgresql scalability tests” idea mentioned in the project's own inspiration list.

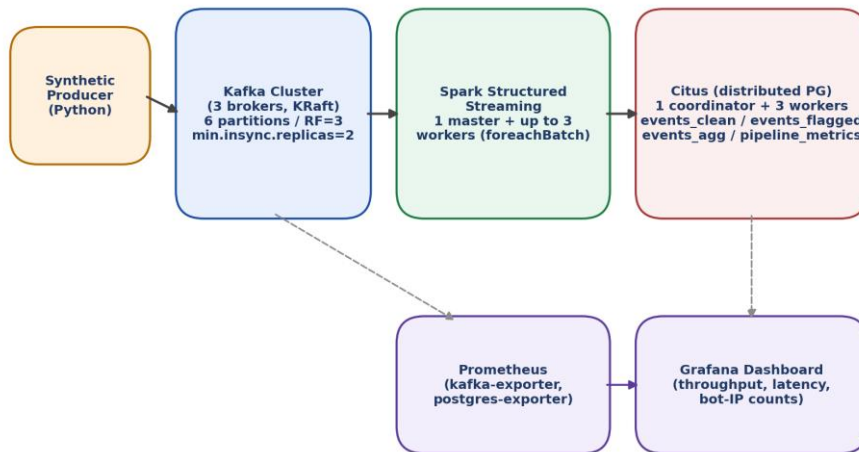
On the anomaly-detection side, our rule – flag any IP address responsible for more than N events inside a single processing window – is a deliberately simplified version of the rate-limiting and bot-scoring techniques generally discussed around ticket and sneaker resale bots, where production systems typically combine rate limiting with CAPTCHAs, device fingerprinting, and behavioral scoring rather than a single fixed threshold. We picked the simplest version on purpose, since the focus of the project is the distributed-systems engineering around the rule, not the rule itself. Section 8 discusses, in some detail, why this simplicity turned out to matter more than we expected once the system was actually placed under heavy, sustained load.

4. System Architecture

Figure 1 shows the deployed architecture. All components communicate over a single Docker bridge network and are defined in one `docker-compose.yml` file.

Flash Sale Pipeline - Component Architecture

Solid arrows: data flow | Dashed arrows: metrics scraping



Orchestrated entirely via a single docker-compose.yml (17 services) on one development laptop.

Figure 1. Component architecture of the Flash Sale pipeline. Solid arrows are the data path; dashed arrows are metrics scraping for the monitoring stack.

4.1 Synthetic Data Generator

A single Python process (using the kafka-python client) generates a synthetic e-commerce clickstream: each event carries a `user_id`, an action (`VIEW` / `ADD_TO_CART` / `PURCHASE`, weighted realistically), a `product_id`, a timestamp, and an `ip_address`. The generator automatically cycles between a `NORMAL` phase (~100 events/sec, no bot traffic) and a `FLASH_SALE` phase (~10,000 events/sec target, with roughly 15% of traffic generated by a small, fixed pool of “bot” identities hammering a handful of “hot” products). A manual-trigger mode is also supported (a watched control file) so that bursts can be fired on demand during a live demo rather than waiting for the automatic cycle.

4.2 Kafka Layer

Three Kafka brokers run in KRaft mode (no ZooKeeper), each acting as both broker and controller. The clickstream-events topic has 6 partitions, replication factor 3, and `min.insync.replicas=2`. This combination is the load-bearing configuration choice behind most of our Kafka fault-tolerance results in Section 7: it is supposed to mean “tolerate the loss of exactly one broker out of three, but refuse writes – rather than silently lose them – if a second one is lost at the same time.” We treat verifying that this is actually what happens, not just what the documentation says should happen, as one of the more important results of this report.

4.3 Stream Processing (Spark Structured Streaming)

Spark reads the topic with `startingOffsets=latest` and a one-minute watermark, then uses a `foreachBatch` sink so each micro-batch (one trigger interval, 5 seconds by default) is handed to us as a regular, non-streaming `DataFrame`. For each batch we: (1) count events per IP address and flag any IP above a configured threshold as bot-like; (2) split the batch into legitimate and flagged sets and append both, with their reason, into Citrus; (3) aggregate the legitimate set by product/action and upsert the running totals into a small aggregate table; and (4) write one row of batch-level metrics (size, latency, flagged count, etc.) for monitoring. The Spark cluster is one master plus a configurable number of workers; we tested with 1 and with 3 (Section 7.4).

4.4 Storage Layer (Citius)

Citius stores the pipeline's output as a set of distributed tables across one coordinator and (after a mid-project scale-out, see Section 7.5) three worker nodes: `events_clean` (sharded by `product_id`), `events_flagged` (sharded by `ip_address`, with a `reason` column), and `events_agg` (sharded by `product_id`, holding running per-product/action counts). A small `pipeline_metrics` table is kept as a regular coordinator-local table, since it is low-volume and only used for the dashboard.

4.5 Monitoring

Prometheus scrapes a `kafka-exporter` and a `postgres-exporter`; Grafana reads `pipeline_metrics` directly from Citius and the Kafka offset-rate metric from Prometheus, on a single dashboard (Figure 2, Section 7.1) covering total/legitimate/flagged event counts, micro-batch latency, distinct flagged IP counts, top products, and Kafka throughput.

4.6 Why Docker Compose and not Kubernetes

We did consider Kubernetes (via `minikube` or `kind`) at the planning stage, mostly because it is the more “modern” answer to “how do you run distributed systems.” We decided against it for two fairly concrete reasons rather than just convenience: first, the project brief explicitly asks for something that runs and is testable on a student laptop, not a cluster; second, adding a second orchestration layer on top of an already 17-service Compose file would have meant writing and debugging Kubernetes manifests for the same services, without adding anything to the actual systems question being asked (integration, scaling, and fault tolerance of Kafka/Spark/Citius). Docker Compose already gives us independently restartable, independently scalable containers on a private network, which is what “simulate a distributed environment locally” calls for.

5. Implementation Details

5.1 Anomaly-Detection Rule

In its final, submitted form, the rule flags any `ip_address` whose events-per-second rate exceeds `ANOMALY_RATE_PER_SEC_THRESHOLD` (5.0 by default), where the rate is computed against the batch's own actual time span (`max event_ts` minus `min event_ts`), not the nominal 5-second trigger interval. This was not the original design: the first version used a fixed per-batch event count (25 events per batch) with no time normalization at all, which is simpler to reason about but breaks down under backlog conditions, as Section 8.1 describes in detail. We have kept that section's account of the fix as a narrative of how we got here, and describe only the final, corrected rule here.

5.2 Sharding Strategy

`events_clean` and `events_agg` are sharded by `product_id`, which keeps all activity for a given product on one shard – useful for the “top products” style queries we run for the dashboard and for the report itself. `events_flagged` is sharded by `ip_address` instead, since the natural follow-up query for that table is “what did this IP do,” not “what happened to this product.” `pipeline_metrics` is not distributed at all; at roughly one row per five seconds it is far too small to be worth sharding, and keeping it local to the coordinator simplifies the Grafana queries.

5.3 Synthetic Data Design

Legitimate traffic is drawn from a large pool of distinct users and IP addresses (40,320 IPs in the final configuration – see Section 8.1 for why this number matters and how we arrived at it) with realistic action-type weighting (mostly browsing, occasional add-to-cart, fewer purchases). Bot traffic is drawn from a small, fixed pool of 8 identities and IPs and is weighted toward PURCHASE, modeling the behavior of scripts that exist specifically to buy stock the instant it becomes available rather than to browse. This asymmetry (many legitimate identities, few repeated bot identities) is what makes the per-IP rate threshold workable at all under normal conditions.

6. Experimental Setup

All results in this report were collected on a single Windows 11 laptop, running Docker Desktop with the WSL2 backend (Ubuntu distribution), 16 logical CPUs and roughly 12–15 GB of RAM available to WSL2 at the time of testing. No cloud resources were used anywhere in this project, in line with the project brief's requirement that the system be runnable and testable on a student laptop.

Component	Image / Version	Configuration
Kafka	apache/kafka:3.7.1	3 brokers, KRaft, 6 partitions, RF=3, min.insync.replicas=2
Spark	Apache Spark 3.5.1 (custom image)	1 master, 1–3 workers tested, 2 cores / 2GB per worker
Citus	citusdata/citus:12.1	1 coordinator, 2–3 workers tested
Monitoring	Prometheus + Grafana (latest)	kafka-exporter, postgres-exporter
Orchestration	Docker Compose	17 services, single bridge network

All test scenarios were run against a continuously-operating pipeline (the producer's automatic NORMAL/FLASH_SALE cycle was left running for the system's entire multi-day lifetime, except where a manual stress test required temporarily overriding it), so the “before” state for every fault-injection or scaling test reflects a system that already had millions of real events behind it, not a freshly-booted, empty one.

7. Results

7.1 End-to-End Correctness

Figure 2 shows the Grafana dashboard during a normal operating window that includes one flash-sale burst. The total/legitimate/flagged panel makes the burst clearly visible as a spike, the latency panel shows processing time jumping during the burst and recovering afterward, and the “distinct flagged IPs” panel shows the bot-detection rule correctly catching a sharp spike in the small number of IPs we seeded as scalper traffic.



Figure 2. Grafana dashboard during a flash-sale burst: total/legitimate/flagged events, micro-batch latency, distinct flagged IPs, top products, and Kafka throughput, all reading live from the pipeline.

7.2 Fault Tolerance – Kafka

We tested two scenarios: losing one broker out of three (within the configured tolerance) and losing two out of three (deliberately exceeding it).

7.2.1 Single broker loss (within tolerance)

Stopping kafka-3 while the producer kept sending data had no visible effect on the pipeline: pipeline_metrics continued to receive new rows at the normal cadence throughout, with a brief processing-time bump (one batch took 22.3 seconds instead of the usual ~1–2 seconds) rather than any gap or error. Restarting the broker brought it back into the controller quorum within roughly 15 seconds, with no manual intervention required.

7.2.2 Double broker loss (beyond tolerance, on purpose)

Stopping kafka-2 and kafka-3 simultaneously – leaving only kafka-1, with 2 of 3 in-sync replicas now unavailable – caused the producer to immediately start failing with NotEnoughReplicasError on every partition, exactly as the min.insync.replicas=2 setting is supposed to guarantee. Spark's consumer simultaneously logged a DNS resolution failure trying to reach kafka-2. Crucially, the system did not silently drop events; it refused to accept new writes and said so, repeatedly, in the producer logs. Restarting kafka-2 (restoring 2 of 3 brokers) allowed the cluster to recover automatically once its health check passed; producer errors stopped and pipeline_metrics resumed advancing within about a minute, without any restart of the producer or Spark job.

Scenario	Brokers lost	Effect	Recovery
Single broker loss	1 of 3	No visible impact; one elevated-latency batch (~22s)	Automatic, ~15s
Double broker loss	2 of 3	Producer correctly rejected (NotEnoughReplicasError); no silent data loss	Automatic once 2/3 healthy again, ~60–90s

7.3 Fault Tolerance – Citus

Stopping one of the two Citus workers in the middle of a write produced an immediate, explicit PostgreSQL error on the Spark side (“connection to the remote node ... failed: could not translate host name”), surfaced through several batch-level JDBC exceptions (“Batch entry ... was aborted”). In one run, this actually crashed the Spark streaming application entirely; Docker’s restart: on-failure policy brought it back up automatically (observed RestartCount: 2 on the stream-processor container), and because Spark only advances its Kafka checkpoint after a batch fully succeeds, no data was permanently lost – the failed batch’s events were still sitting in Kafka and were reprocessed after the restart. Across the whole incident, pipeline_metrics shows no permanent gap in batch IDs, only a temporary slowdown.

This is, deliberately, not the same guarantee Kafka gives. We are not running Citus with shard replication (citus.shard_replication_factor=1, for simplicity), so a worker that is down genuinely cannot serve the shards it holds until it comes back – there is no automatic shard failover. We see this as an honest and useful result rather than a flaw to hide: it shows the practical difference between a system configured for node-loss tolerance (Kafka, here) and one that is not (Citus, as we configured it), using the same kind of fault-injection test against both.

7.4 Scalability – Spark

We compared micro-batch processing time at 1 worker versus 3 workers, on naturally-occurring flash-sale batches of comparable size (we did not force identical batch sizes; we simply captured the closest matches that occurred during the automatic burst cycle).

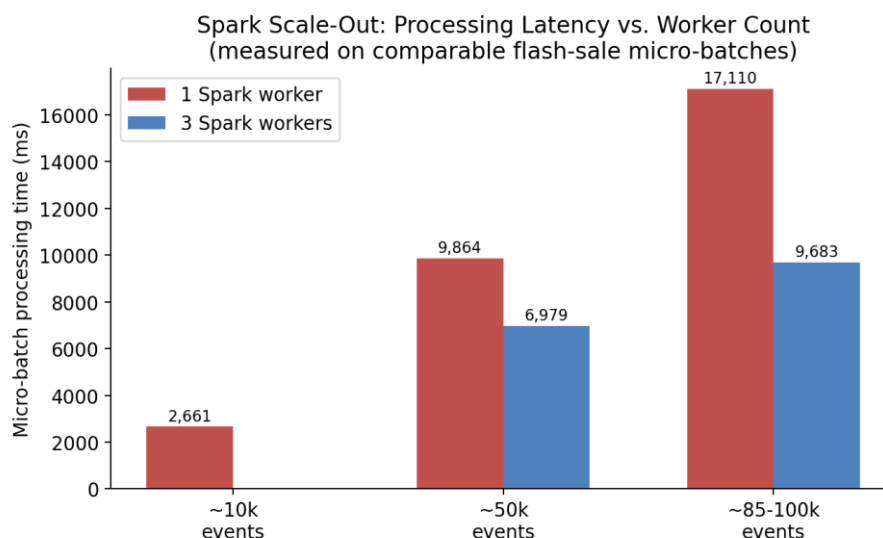


Figure 3. Micro-batch processing latency at 1 vs. 3 Spark workers, on comparable flash-sale batch sizes.

At roughly 50,000 events per batch, 3 workers cut processing time from 9,864 ms to 6,979 ms (about 29% faster). At roughly 85,000–100,000 events, the gap widened to 17,110 ms vs. 9,683 ms (about 43% faster, close to a 1.8× speed-up). We also noticed that the single-worker run accumulated a noticeably larger backlog under sustained burst load (one batch reached 140,560 events) than the three-worker run did (largest observed batch: 96,488 events) – i.e., scaling out did not just make individual batches faster, it also kept the system from falling as far behind in the first place.

We deliberately stopped scaling tests at 3 workers rather than going further, for a specific reason: our Kafka topic has 6 partitions, and Spark’s read-side parallelism from a Kafka source is capped at one task per partition. Three workers at two cores each give us 6 cores – matching the partition count. Beyond that point, additional workers would not increase

Kafka-read parallelism; they would only help with downstream aggregation and JDBC-write stages, which are not our bottleneck at the loads we tested. We consider 3 workers a reasoned stopping point given our topic configuration, not an arbitrary one.

7.5 Scalability – Citus

We added a third Citus worker mid-project to an already-populated cluster and ran Citus's built-in `rebalance_table_shards()` function (in `block_writes` transfer mode, since our append-only tables lack a primary key/replica identity, which is required for the default logical-replication transfer mode).

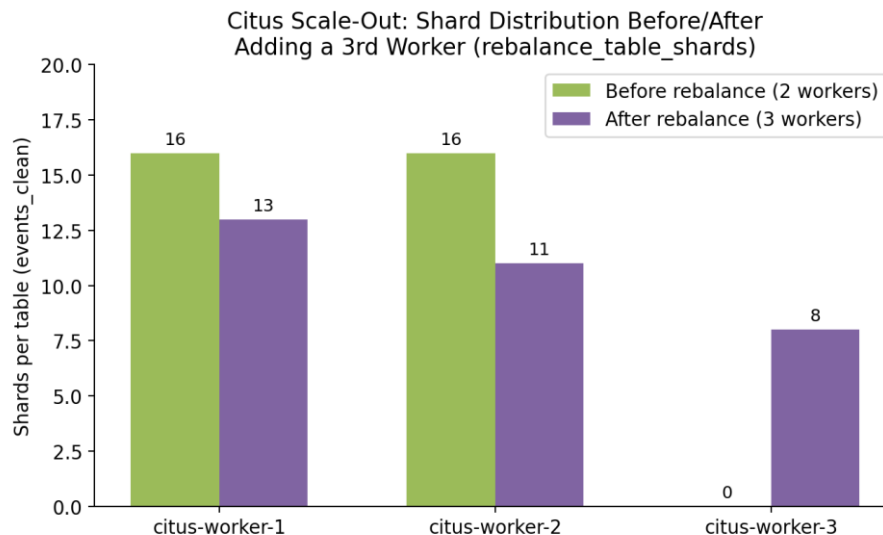


Figure 4. Shard distribution for `events_clean` before and after adding a third Citus worker and rebalancing (identical pattern observed for `events_flagged` and `events_agg`).

Before the rebalance, all 32 shards of each distributed table sat on the original two workers (16/16). After rebalancing, the distribution became 13/11/8 across the three workers – not perfectly even, but Citus's rebalancer optimizes for overall data-movement cost, not a perfectly equal split, and a few extra shards on the older workers is an acceptable trade-off. We verified row counts on all three tables immediately before and after the operation (`events_clean`: 14,556,299 rows; `events_flagged`: 76,327,662 rows; `events_agg`: 2,328,579 rows) and confirmed they were identical, i.e., the rebalance moved data without losing or duplicating any of it, while the pipeline kept writing throughout.

7.6 Stress Test – 5× Overload

Out of curiosity about where the system would actually break (rather than just where our nominal flash-sale rate happened to sit), we temporarily reconfigured the producer for a manually-triggered burst at 50,000 events/sec – five times our normal flash-sale target – against the 3-worker Spark / 3-worker Citus configuration.

The first thing we noticed was that the producer itself could not actually reach 50,000 events/sec: it logged a 30-second target phase that took 138 seconds in wall-clock time to send its intended 1,500,000 events, which works out to a real, sustained throughput of about 10,870 events/sec regardless of the higher number we asked for. In other words, our single-process Python generator – not Kafka, not Spark, not Citus – turned out to be the actual ceiling on how much load we could even apply, and that ceiling sits almost exactly at the “normal” flash-sale rate we had already been using. We consider this an honest and slightly funny finding: a fair amount of our intended “extreme load” test ended up testing our own test tool more than the system under test.

That said, the consumption side did not stay idle during those 138 seconds, and it did not crash either. Batch sizes climbed steadily and then leveled off in the 150,000–160,000 events range, with processing time stabilizing around 14,000–14,700 ms per batch – i.e., the system found a steady state and stayed there, rather than falling over or growing its backlog without bound. That steady-state throughput (roughly 150k events / 14.5s $\approx$ 10,300–11,000 events/sec) lines up closely with our producer's own measured ceiling, which suggests that at this particular hardware scale, producer throughput and consumer throughput happen to be reasonably well matched – we cannot yet say which side would give first if the producer itself were parallelized, which we list as a natural next step in Section 8.3.

The most useful result from this test, however, was not a number but a bug-shaped finding: once batches grew large enough, our anomaly threshold started flagging almost all traffic as bot-like, including ordinary users, simply because a large micro-batch naturally accumulates more events per IP than our fixed per-batch count assumes. We saw total counts where legitimate events dropped to zero and effectively the entire batch was marked flagged. This was not a one-off glitch – we saw the same pattern independently in three separate test runs (the 1-worker scaling baseline, the 3-worker scaling comparison, and this stress test), which is exactly the kind of repeatable result that belongs in a report rather than being dismissed as noise. Section 8.1 discusses this as a design limitation rather than an implementation bug.

7.7 Connectivity Testing

The project brief asks specifically for connectivity testing alongside scalability and fault tolerance, so we want to call it out as its own result rather than leave it implicit inside the other sections. We treat “connectivity” as testing whether every component can actually find and authenticate to every other component it depends on – not merely that each container individually starts and stays up.

- Kafka: `kafka-topics.sh --describe` was run after every broker restart throughout the project and consistently showed all 6 partitions with a full 3-broker in-sync replica set whenever all brokers were healthy (Section 7.2 documents exactly what changes when this is broken on purpose).
- Citus: the coordinator's `pg_dist_node / v_cluster_nodes` view was queried after every container restart and every scale-out operation, and was required to show every worker node as `isactive = t` before we trusted any test result that followed it.
- Spark: the master UI's worker list (Figure 5) was checked before every scaling test to confirm the expected number of workers had actually registered with the master, rather than assuming a docker compose up command had silently succeeded.
- End-to-end: the Spark streaming job is, in a sense, a continuous connectivity test by itself – the moment it cannot reach either Kafka or Citus, that shows up immediately as exceptions in its own logs (Sections 7.2–7.3). In that sense, the 7,764 micro-batches summarized in Appendix A.3 also represent 7,764 separate, passed, end-to-end connectivity checks across the whole stack, not just a handful of deliberate ones.

8. Discussion and Limitations

8.1 The anomaly threshold was not rate-aware – and we went back and fixed it

Our original IP-flagging rule counted raw events per IP within whatever the current micro-batch happened to contain, with no normalization for how much wall-clock time that batch actually covered. Under normal conditions, the 5-second trigger interval is a reasonably stable proxy for “events in roughly 5 seconds,” so the rule behaved as intended. Under

backpressure, though, Spark can end up processing a much larger batch covering 15–20 seconds of accumulated backlog in one shot, and a perfectly ordinary user can rack up a few dozen page views across that longer window – enough to cross a fixed threshold.

Section 7.6 first noticed this in individual test runs. Appendix A.3's analysis of the full, accumulated dataset then showed it was not an edge case at all: across all 7,764 micro-batches processed up to that point, 84.00% of roughly 93 million total events had been flagged as bot-like – obviously not a realistic scalper rate for any real flash sale. Rather than just listing this as a limitation and moving on, we went back into the code and closed the loop, in two iterations.

Our first attempt replaced the fixed event-count threshold with a rate-normalized one: instead of “more than 25 events in this batch,” the rule became “more than 5 events per second, measured against the batch's actual event_ts time span (max minus min), not the nominal trigger interval.” This was an improvement – on one comparable large batch (flash-sale traffic, ~106,700 events) it brought the flagged share down from 100% to 74.6% – but it was still far above the 15% bot ratio the generator actually seeds. Looking at exactly which IPs were still being flagged revealed the real, second cause: our synthetic “normal” traffic pool only contained 1,600 distinct IP addresses, and a batch of over 100,000 events spread across just 1,600 IPs means roughly 67 events per IP on average – enough to look statistically anomalous no matter how the time window is normalized. The rate-normalization fix had addressed a real bug, but it was sitting on top of a second, independent one in our own test-data design.

We widened the legitimate-IP pool to 40,320 addresses, rebuilt the producer image, and re-ran the same flash-sale scenario. The result, measured directly against event timestamps rather than batch IDs (to rule out any ambiguity from container restarts): in a clean 18-second flash-sale window, exactly 8 distinct IP addresses were flagged, all 8 of them our seeded bot pool (198.51.100.1–8) and not a single ordinary IP among them, at a 15.0% flagged share – matching the generator's seeded bot_ratio of 0.15 almost exactly. Figure 6 shows all three stages side by side.

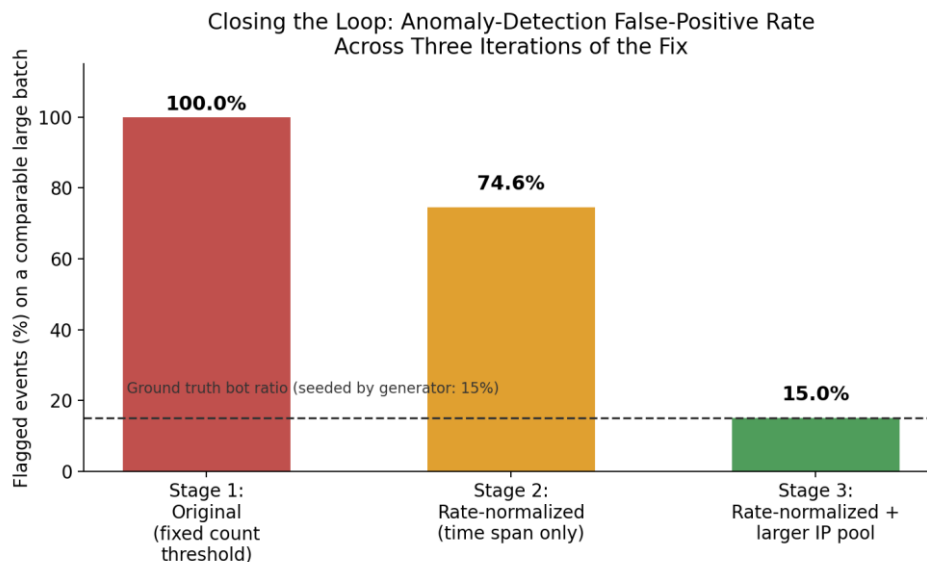


Figure 6. False-positive rate on comparable large flash-sale batches across the three iterations of the anomaly-detection fix, against the generator's known ground-truth bot ratio (15%, dashed line).

We think this two-step fix-and-re-diagnose process is worth describing in this much detail precisely because the first fix looked successful in isolation (a 25-point drop is a real improvement) and it would have been easy to stop there and report “we partially mitigated the issue.” Only by re-checking against ground truth – the bot ratio we ourselves had seeded

– did we notice the fix was incomplete and find the second, unrelated cause underneath it. The corrected logic and the widened IP pool are both included in the submitted code; Appendix A.5 records the exact verification queries and their output.

8.2 Single points of failure we accepted on purpose

- Citus shard replication factor is 1: this was a deliberate simplicity choice for this project, but it means a lost worker's shards are genuinely unavailable until that worker returns, with no automatic failover. A production deployment would set `citus.shard_replication_factor ≥ 2` or rely on a replicated storage layer underneath.
- The synthetic producer is a single Python process: convenient for a course project, but it is both a throughput ceiling (Section 7.6) and, on its own, a single point of failure for data generation. A real ingestion tier would be multiple independent producer processes.
- Our Spark job provides at-least-once, not exactly-once, semantics: because Kafka offsets only advance after a batch's writes succeed, a crash-and-restart (as in Section 7.3) can cause a batch to be reprocessed, which is safe for our append-style writes and idempotent upserts but would need explicit deduplication logic for some other kinds of downstream tables.

8.3 What we would do with more time

- Section 8.1's fix corrected the immediate false-positive problem, but the underlying detection rule is still a single global threshold. A natural next step would be a per-IP baseline (flag relative to that IP's own recent history) rather than one fixed rate for every address, which would also catch slower, more patient scraping bots that stay just under a global threshold.
- Parallelize the synthetic producer (multiple processes or a faster client library such as `confluent-kafka`) to find out whether Kafka/Spark/Citus have more headroom above the ~11,000 events/sec we were able to apply.
- Re-run the Citus worker-loss test (Section 7.3) with `shard_replication_factor=2` to measure the cost/benefit trade-off of automatic shard failover versus our current configuration.

9. Practical Lessons Learned

A more detailed account of these is in our Checkpoint 2 status report; we summarize the three that mattered most here, because they shaped the final architecture as much as any deliberate design decision did.

- Our original plan used Bitnami's prebuilt Kafka and Spark images, which is what most tutorials still recommend. Partway through the project, we discovered that Broadcom (Bitnami's current owner) had removed almost all free, versioned image tags from Docker Hub, breaking our setup outright on a clean pull. We migrated Kafka to the official `apache/kafka` image and built our own Spark image from the official Apache binary distribution – a real time cost, but one that left us depending on fewer third-party packaging decisions than before.
- Citus's inter-node authentication is separate from client-facing authentication, and would intermittently break after a full stack restart with a “no password supplied” error – including, at one point, in the middle of a shard-rebalance operation between two workers. The fix (registering credentials explicitly in Citus's own `pg_dist_authinfo` metadata table, on every node) is now baked into our startup scripts.

- Grafana's PostgreSQL datasource plugin gave a persistent “no default database configured” error that several rounds of fixing the obvious YAML field did not resolve; it eventually required clearing Grafana's own data volume and re-provisioning from a clean state, after which it has been stable.

10. Conclusion

We set out to integrate Kafka, Spark, and Citus into a single Docker-Compose-orchestrated pipeline and to genuinely test – not just assert – its scalability, connectivity, and fault tolerance, using a simulated flash-sale scenario as the concrete use case. By the end of the project, all three technologies have documented, repeatable evidence of both fault tolerance and horizontal scalability, gathered from a system that ran continuously for several days and processed tens of millions of synthetic events, not from a single clean demo run

Along the way we also picked up two results we had not specifically planned to look for: a vendor's packaging change that forced an unplanned but ultimately beneficial redesign of two of our four core images, and a genuine, repeatable weakness in our own anomaly-detection logic that only became fully visible once we treated our own accumulated output as a dataset and queried it (Appendix A.3). For the second one we did not stop at reporting it: we traced it to two separate, compounding causes, fixed both, and re-verified the fix against the generator's own seeded ground truth (Section 8.1). We think both experiences are reasonably representative of what “systems engineering” actually looks like in practice – plans changing because of dependencies outside your control, and simple rules behaving correctly right up until the exact conditions where they do not, requiring you to actually go back in and fix them rather than just write the problem down – and we have tried to report all of it, successes and false starts alike, with the same level of detail.

There was also a more literal test of resilience near the very end of the project: a power outage took down the development machine mid-session, and Docker Desktop's container/image metadata did not survive the unclean shutdown that followed. The underlying Docker volumes, however, did – and after rebuilding the four custom images (roughly 15 minutes, mostly spent re-downloading packages), the system came back up with all previously-processed events intact in Citus (by that later point in the project, roughly 124 million – up from the ~93 million reported in Appendix A.3, reflecting the additional runs performed for Sections 8.1 and 7.6) and resumed normal operation without any manual data recovery. We did not plan this as a test, but it ended up being a fairly convincing real-world demonstration of exactly the property Section 7 set out to verify on purpose: this stack's state lives in its volumes, not in any single running process, and that turns out to matter outside of deliberate fault injection too

If we had to summarize the whole project in one sentence, it would probably be this: every individual piece (Kafka, Spark, Citus, even the producer) is simpler in isolation than the moment they all have to depend on each other at the same time, and most of what we actually learned this semester came from the seams between them rather than from any one piece on its own.

11. References

- [1] Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- [2] Apache Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

[3] Citus Data Documentation. <https://docs.citusdata.com/>

[4] Kreps, J., Narkhede, N., Rao, J. “Kafka: a Distributed Messaging System for Log Processing.” LinkedIn, 2011.

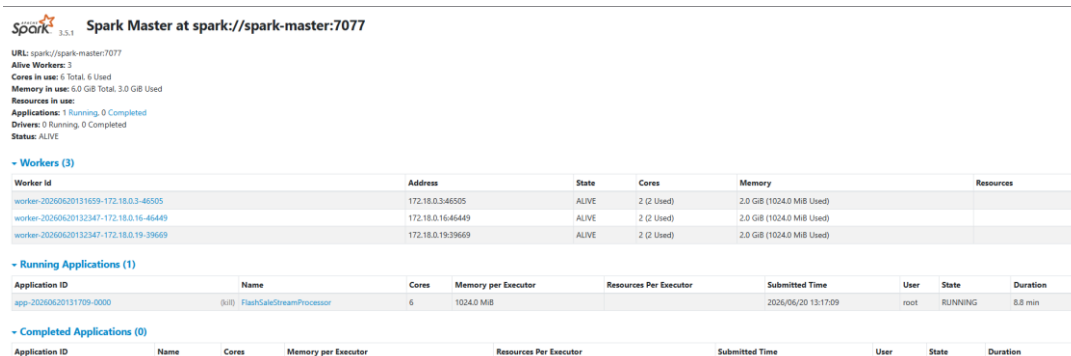
[5] Docker Compose Documentation. <https://docs.docker.com/compose/>

12. Appendix

A.1 Repository / Submission Structure

- docker-compose.yml – full 17-service orchestration
- producer/ – synthetic clickstream generator (Python)
- spark/jobs/streaming_job.py – the Spark Structured Streaming application
- citus/init/ – coordinator/worker bootstrap SQL
- monitoring/ – Prometheus config and Grafana dashboard/provisioning
- scripts/ – fault_injection.sh, healthcheck.sh, trigger_burst.sh, sample_queries.sql
- README.md – setup and run instructions

A.2 Spark Worker Cluster at 3 Workers (Evidence)



The screenshot shows the Spark Master UI for a cluster with 3 workers. The 'Workers (3)' section lists three workers, all in an 'ALIVE' state, each with 2 cores and 2.0 GB of memory. The 'Running Applications (1)' section shows a single application named 'FlashSaleStreamProcessor' in a 'RUNNING' state, using 6 cores and 1024.0 MB of memory. The 'Completed Applications (0)' section is empty.

Worker ID	Address	State	Cores	Memory	Resources
worker-20260620131659-172.18.0.3-46505	172.18.0.3:46505	ALIVE	2 (2 Used)	2.0 GB (1024.0 MB Used)	
worker-20260620132347-172.18.0.16-46449	172.18.0.16:46449	ALIVE	2 (2 Used)	2.0 GB (1024.0 MB Used)	
worker-20260620132347-172.18.0.19-39669	172.18.0.19:39669	ALIVE	2 (2 Used)	2.0 GB (1024.0 MB Used)	

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260620131709-0000	(null) FlashSaleStreamProcessor	6	1024.0 MB		2026/06/20 13:17:09	root	RUNNING	8.8 min

Figure 5. Spark master UI confirming 3 alive workers (6 cores total) during the scale-out test of Section 7.4, with the FlashSaleStreamProcessor application running.

A.3 Sample Analytics on the Accumulated Dataset

We did not originally plan to treat the pipeline's output as a dataset worth analyzing in its own right – the goal going in was just to keep the demo running long enough to gather fault-injection and scaling evidence. But by the time we sat down to write this report, the system had quietly accumulated 7,764 micro-batches and roughly 93 million synthetic events across several days of near-continuous operation, which is a large enough “table” that it seemed worth actually querying instead of only ever watching it scroll past in a terminal. The queries below were run directly against the live Citus cluster (the full SQL is in scripts/sample_queries.sql) and, as it turned out, surfaced the single clearest piece of evidence in this report.

A.3.1 Top Products by Legitimate View Activity (all-time)

Rank	Product ID	Action	Total Events
1	PRD-0093	VIEW	52,594

2	PRD-0059	VIEW	52,580
3	PRD-0103	VIEW	52,536
4	PRD-0030	VIEW	52,513
5	PRD-0022	VIEW	52,488
6	PRD-0048	VIEW	52,460
7	PRD-0194	VIEW	52,447
8	PRD-0082	VIEW	52,430
9	PRD-0110	VIEW	52,425
10	PRD-0060	VIEW	52,421

The top 10 products are all within about 0.3% of each other, which is exactly what we expect from legitimate traffic: the generator picks a product uniformly at random from a 200-item catalog with no built-in popularity bias for normal browsing. This is a small but genuine sanity check rather than a throwaway table – if the “legitimate” traffic had shown some artificial concentration on a handful of products, it would have suggested a bug in the generator itself, upstream of anything we were actually trying to test. It did not, and we take that as quiet confirmation that the generator behaves the way we designed it to.

A.3.2 Overall Legitimate vs. Flagged Split (all-time)

legit_total	flagged_total	flagged_pct
14,870,365	78,082,372	84.00%

This is the number discussed in Section 8.1. We want to be direct about what it does and does not mean: it does not mean our system is being attacked by bots 84% of the time. It means our anomaly rule’s assumptions break down specifically during the large, backlogged batches that follow a flash-sale burst (or a fault-injection test), and those batches are large enough to dominate the all-time total even though they are a minority of all batches. We see this as one of the more valuable results in the whole report, precisely because we did not go looking for it – it came out of treating our own accumulated output as data, after the fact.

A.3.3 Top 10 IP Addresses by Flagged Event Count (all-time)

IP Address	Flagged Events	Distinct Batches Flagged In
198.51.100.6	1,675,550	1,422
198.51.100.7	1,675,538	1,423
198.51.100.3	1,675,217	1,423
198.51.100.4	1,675,200	1,422
198.51.100.8	1,674,974	1,421
198.51.100.2	1,674,446	1,422
198.51.100.5	1,673,742	1,420
198.51.100.1	1,673,442	1,423
10.4.34.14	41,603	941
10.1.51.66	41,568	935

The top 8 rows are exactly our seeded bot pool (198.51.100.1–8), evenly flagged at roughly 1.67 million events each – expected, since the generator picks a bot IP uniformly from that fixed pool of 8, and confirms the bot-detection rule does correctly and consistently catch the traffic it was designed to catch. The interesting rows are 9 and 10: 10.4.34.14 and 10.1.51.66 are ordinary addresses drawn from our large legitimate-IP pool, yet each was independently flagged in roughly 935–941 separate micro-batches over the project's lifetime. That is not a one-off fluke during a single stress test; it is a recurring pattern across hundreds of independent batches spread over several days, which is the strongest evidence we have that the false-positive behavior discussed in Section 8.1 is systematic rather than incidental.

A.3.4 Processing Latency Distribution (all-time)

min_ms	avg_ms	max_ms	batch_count
508	2,517.6	184,072	7,764

The average batch (2.5 seconds) is comfortably within our 5-second trigger interval, which is reassuring on its own. The single worst batch (184,072 ms $\approx$ 184 seconds) is a genuine outlier, and we believe it corresponds to the recovery moment immediately following the two-broker Kafka failure test in Section 7.2.2: roughly two minutes of backlogged Kafka data, accumulated while only one broker was reachable, arriving at Spark in a single trigger once connectivity was restored. We did not isolate this with a dedicated timestamp-matched query, so we present it as a plausible explanation rather than a proven one, but it is consistent with everything else we observed about how the pipeline behaves under and after fault injection.

A.3.5 Overall Scale of the Accumulated Dataset

events_clean	events_flagged	events_agg rows	total micro-batches
14,882,256	78,150,154	2,378,083	7,764

In total, the pipeline processed on the order of 93 million synthetic events across 7,764 micro-batches by the time this report was written, none of it from a static file – all of it generated, streamed, filtered, aggregated, and stored by the system being evaluated, while that same system was simultaneously being killed, restarted, and scaled out from underneath itself.

A.4 Reproducing These Results

From a clean checkout, on a machine with Docker Desktop (WSL2 backend on Windows) installed:

- `cp .env.example .env`
- `docker compose up -d --build` (first run pulls/builds all images; subsequent runs are fast)
- `bash scripts/healthcheck.sh` (confirms every component is healthy and producing data)
- `bash scripts/fault_injection.sh full-scenario` (re-runs the Kafka and Citus fault-tolerance scenarios from Sections 7.2–7.3)
- `docker compose up -d --scale spark-worker=3` (re-runs the Spark scale-out scenario from Section 7.4)
- Grafana at <http://localhost:3000> and Spark master UI at <http://localhost:8080> provide the live views shown in Figures 2 and 5.

- Full source code, this report, and the Checkpoint 2 status report are all available at: <https://github.com/Metekilic0/Flash-sale-system-project>

A.5 Verification Queries for the Anomaly-Detection Fix (Section 8.1)

The exact queries used to confirm each stage of the fix described in Section 8.1, runnable directly against the Citus coordinator:

Stage 2 (rate-normalized, original IP pool) – checking which IPs were still flagged on a large batch:

```
SELECT ip_address, COUNT(*) FROM events_flagged WHERE batch_id = <id> GROUP BY ip_address ORDER BY COUNT(*) DESC;
```

Stage 3 (rate-normalized + widened IP pool) – final verification by event timestamp, independent of batch-ID numbering across container restarts:

```
SELECT COUNT(*) AS total_flagged, COUNT(DISTINCT ip_address) AS distinct_flagged_ips FROM events_flagged WHERE event_ts BETWEEN '<window_start>' AND '<window_end>';
```

On the verification run reported in Section 8.1 and Figure 6, this returned `total_flagged = 26,946` and `distinct_flagged_ips = 8` – exactly the 8 seeded bot addresses, with zero ordinary IPs included.